

Documentation for DP-Based Partitioning and Offline Scheduling Codebase

Your Name

February 18, 2025

Contents

1	Introduction & Motivation	2
1.1	Overview	2
1.2	Motivation	2
1.3	Scope	2
2	Architecture Overview	2
2.1	System Diagram	2
2.2	Component List and Responsibilities	3
3	Detailed Component Descriptions	3
3.1	Node Class	3
3.2	Utility Functions	3
3.3	Profiler Class	3
3.4	Metric Interface and HPCMakespanMetric Class	4
3.5	Task Class	4
3.6	Stage Class	4
3.7	Taskset Class	5
3.8	Evaluator Class	5
4	Overall System Flow (Up to Evaluator Testing)	5
5	Future Work: Runtime Script	6
6	Conclusion	6

1 Introduction & Motivation

1.1 Overview

This codebase implements a DP-based partitioning and offline scheduling framework for accelerating DNN inference. The system partitions a DNN’s FX graph into stages, each of which is scheduled on an individual node (CPU or GPU). The primary objective is to minimize the maximum execution time (or makespan) by balancing the load across heterogeneous compute nodes.

1.2 Motivation

- **Bottleneck Minimization:** The approach focuses on minimizing the blocking time of the slowest stage.
- **Extensibility:** A modular metric interface allows the incorporation of various performance measures (e.g., execution time, memory usage, arithmetic intensity).
- **Adaptability:** Designed to work from resource-constrained environments to large-scale data centers.

1.3 Scope

This document covers the components from node management and profiling up to the evaluator testing phase. A runtime script (for processing inputs in real time) is planned for future development and will follow a similar procedure.

2 Architecture Overview

2.1 System Diagram

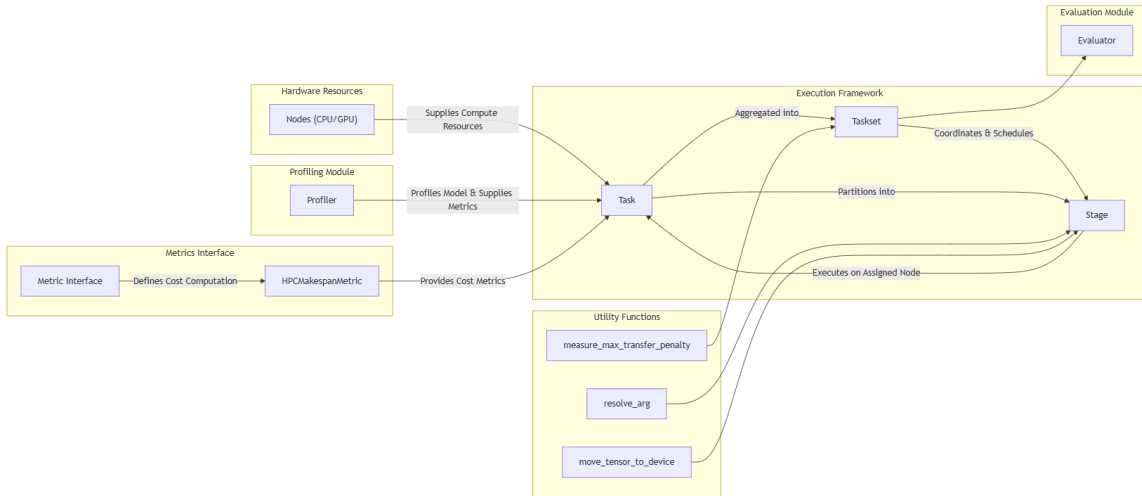


Figure 1: System Design Diagram

2.2 Component List and Responsibilities

- **Node:** Manages execution on individual CPU cores/GPUs, maintains a task queue, and handles CPU affinity and device context.
- **Utility Functions:** These functions support the framework by:
 - **resolve_arg:** Recursively resolves FX node arguments using a mapping of node outputs.
 - **move_tensor_to_device:** Transfers tensors (or nested containers of tensors) to a specified device.
 - **measure_max_transfer_penalty:** Measures the worst-case data transfer time between available nodes.
- **Profiler:** Uses PyTorch’s profiling tools to trace the FX graph, instrument key operations, and collect per-layer metrics (execution time, memory usage).
- **Metric Interface and HPCMakespanMetric:** Define a standard interface for computing per-layer and per-task cost metrics; HPCMakespanMetric computes cost based on execution time.
- **Task:** Encapsulates a single DNN inference execution; stores the model, input, profiling records, and uses DP-based partitioning to create stages.
- **Stage:** Represents a contiguous block of FX nodes scheduled on a node; manages data transfer and stage execution.
- **Taskset:** Aggregates tasks, schedules them on available nodes, and computes system-wide metrics (makespan, throughput, utilization).
- **Evaluator:** Compares naive (sequential) and parallel (DP-partitioned) execution strategies, validates outputs, and computes performance metrics.

3 Detailed Component Descriptions

3.1 Node Class

Purpose & Responsibilities:

- Represents a compute unit (CPU core or GPU with an associated core).
- Manages a task queue and runs a dedicated worker thread.
- Handles setting and resetting the device context (CPU affinity, GPU device).

Key Methods:

- **assign_task:** Enqueues a function for execution.
- **stop:** Signals the worker thread to terminate.
- **discover_nodes:** Discovers and returns a list of available nodes.

3.2 Utility Functions

resolve_arg: Recursively resolves FX node references in function arguments using a dictionary of node outputs.

move_tensor_to_device: Transfers tensors (or nested containers of tensors) to a designated device.

measure_max_transfer_penalty: Measures the worst-case time required to transfer a tensor between nodes, providing an estimate of communication overhead.

3.3 Profiler Class

Purpose & Responsibilities:

- Instruments the model using FX and PyTorch’s profiler.
- Wraps key operations (functions, modules, methods) with profiling contexts.

- Collects and caches per-layer execution and memory usage data.
- Saves profiling data in CSV format.

Key Methods:

- `profile_model`: Profiles a model on a given node with warmup and active iterations.
- `_clone_model_safely`: Safely clones the model.
- `_trace_and_instrument_model`: Instruments the FX graph by wrapping nodes with profiling contexts.
- `_process_profiler_data`: Aggregates events and updates the profiling database.

3.4 Metric Interface and HPCMakespanMetric Class

MetricInterface (Abstract Base Class):

- Defines the protocol with two methods:
 - `compute_task`: Computes an overall metric for a task.
 - `compute_layer`: Computes a cost metric for a single layer on a given node.

HPCMakespanMetric:

- Implements MetricInterface.
- Uses the overall forward pass time as the task-level metric.
- Computes per-layer cost based on the recorded “Total Execution Time (us)”.

3.5 Task Class

Purpose & Responsibilities:

- Represents an instance of DNN inference.
- Holds the model, input data, profiling records, and computed partitions.
- Uses DP-based partitioning to split the FX graph into stages.
- Manages stage execution and collects the final output.

Key Methods:

- `_initialize_dag`: Extracts FX nodes from the model.
- `populate_profile_records`: Retrieves profiling data and stores it.
- `run_offline_partition_makespan`: Implements the DP algorithm for partitioning.
- `get_forward_pass_time`: Computes the overall forward pass time.
- `print_stage_allocations`: Prints the mapping of stages to nodes.

3.6 Stage Class

Purpose & Responsibilities:

- Represents a contiguous block of FX nodes (a stage) scheduled on a particular node.
- Handles data transfer to the appropriate device.
- Executes its assigned nodes sequentially and records execution and transfer times.
- Manages dependencies between stages.

Key Methods:

- `run_stage`: Executes the stage and handles data movement.
- `add_dependency` and `add_dependent`: Manage dependency relationships.

3.7 Taskset Class

Purpose & Responsibilities:

- Aggregates multiple Task objects.
- Initiates DP-based partitioning for each task.
- Schedules and executes tasks in parallel across available nodes.
- Computes system-wide metrics (makespan, throughput, utilization).

Key Methods:

- `execute_all`: Runs all tasks concurrently.
- `_execute_task`: Handles the execution flow for an individual task.
- `_calculate_metrics`: Computes overall performance metrics.

3.8 Evaluator Class

Purpose & Responsibilities:

- Compares two execution strategies:
 - Naive (sequential) execution.
 - Parallel execution using DP-based partitioning.
- Validates correctness by comparing outputs.
- Computes performance metrics such as speedup, throughput, and task completion rate.

Key Methods:

- `run_naive_execution`: Executes tasks sequentially.
- `run_parallel_execution`: Executes tasks using the DP-partitioned taskset.
- `compare_outputs`: Compares outputs from the two strategies.
- `analyze_speedup_throughput`: Computes and prints performance comparisons.

4 Overall System Flow (Up to Evaluator Testing)

1. **Initialization:** The system sets up the environment (e.g., random seed), discovers available nodes, and instantiates the Profiler.
2. **Profiling:** Each task (constructed from test models like `SimpleCNN` and `PretrainedResNet18`) is profiled across selected nodes. The profiler collects per-layer execution time and memory usage, storing these in a CSV file.
3. **Dynamic Programming Partitioning:** Each Task runs the `run_offline_partition_makespan` method:
 - The FX graph is extracted.
 - A cost matrix is computed from the profiling data.
 - A DP table is constructed to minimize the maximum stage cost.
 - Stage objects are created and assigned to the best nodes.
4. **Taskset Creation:** The tasks and nodes are aggregated into a Taskset, which further computes overall metrics (utilization, makespan, throughput).
5. **Evaluator Testing:** The Evaluator (or a similar testing script) executes both the naive and parallel approaches:
 - Execution times are measured.
 - Outputs are compared to validate correctness.
 - Performance metrics (speedup, throughput) are computed.

5 Future Work: Runtime Script

- The same overall procedure (profiling, DP partitioning, scheduling, execution) will be adapted for runtime operation.
- A runtime script will be developed that processes input batches or streams in real time.
- For now, the documentation and codebase focus on offline evaluation. The runtime module will follow a similar architecture and be documented in future releases.

6 Conclusion

- **Summary:** This codebase implements a DP-based partitioning framework for DNN inference acceleration. The modular design (Nodes, Profiler, Metric, Task, Stage, Taskset, Evaluator) enables efficient scheduling across heterogeneous devices.
- **Adaptability:** Designed for both resource-constrained environments and large-scale HPC systems, the framework uses per-layer profiling and an extensible metric interface.
- **Next Steps:** Future work will include the development of a runtime script for real-time processing.